

Guia do Aluno: Projeto Final em Go

Construindo uma Blockchain com Proof of Work

Neste projeto final vocês vão construir uma **blockchain educacional utilizando Go**.

A proposta não é simplesmente copiar um código pronto. O objetivo é juntar vários assuntos que vimos durante o curso e utilizá-los para resolver um problema maior.

Vocês vão trabalhar com:

- Structs
- Slices
- Funções
- Ponteiros
- Loops
- Condicionais
- Tratamento de erros
- Arquivos
- JSON
- SHA-256
- Persistência de dados

E, ao mesmo tempo, aprender alguns conceitos fundamentais de blockchain:

- Blocos
- Transações
- Hash
- Genesis Block
- Previous Hash
- Nonce
- Proof of Work
- Mineração
- Ledger
- Validação

Minha principal recomendação é: **não tentem construir tudo de uma vez**.

Vamos dividir o problema em pequenas partes.

1. O que vocês deverão construir

Ao final do projeto teremos um programa executado pelo terminal com um menu semelhante a:

```
=====
BLOCKCHAIN EM GO
=====
```

```
1 - Criar nova transação
2 - Minerar bloco
3 - Mostrar blockchain
4 - Validar blockchain
5 - Mostrar último bloco
6 - Informações da blockchain
7 - Mostrar transações pendentes
0 - Sair
```

O usuário poderá criar uma transação:

```
Origem: João
Destino: Maria
Valor: 500
```

Ela ficará aguardando mineração.

Depois um minerador poderá criar um novo bloco.

```
Minerador: Antonio
```

```
Minerando...
```

```
Nonce encontrado: 48372
Tentativas: 48373
```

```
Hash:
0000837ac912...
```

Esse bloco será adicionado à blockchain e salvo em um arquivo:

```
ledger.json
```

O mais importante é que **a blockchain não poderá desaparecer quando o programa for fechado.**

Se tivermos:

```
Genesis
  ↓
Bloco 1
  ↓
Bloco 2
  ↓
Bloco 3
```

e fecharmos o programa, na próxima execução ele deverá recuperar esses blocos e continuar:

```
Genesis
  ↓
Bloco 1
  ↓
Bloco 2
  ↓
Bloco 3
  ↓
Bloco 4
```

Esse é um dos principais objetivos do trabalho.

2. Antes de começar: preparem o ambiente

Vocês precisam ter o Go instalado.

Utilizem a documentação oficial:

Getting Started with Go

<https://go.dev/doc/tutorial/getting-started>

Depois da instalação, abram o PowerShell e executem:

```
go version
```

Se aparecer a versão instalada, o ambiente está pronto.

Criem a pasta:

```
mkdir blockchain-go
```

Entrem nela:

```
cd blockchain-go
```

Inicializem o módulo:

```
go mod init blockchain-go
```

Criem:

```
main.go
```

E façam um teste simples:

```
package main

import "fmt"

func main() {
    fmt.Println("Iniciando projeto Blockchain!")
}
```

Executem:

```
go run .
```

Se funcionar, podemos começar.

3. Primeira dica: revisem Structs

Nossa blockchain será formada por blocos.

Por isso precisamos representar um bloco dentro do Go.

Antes de pensar em blockchain, façam um teste simples:

```
type Produto struct {
    Nome    string
    Preco   float64
}
```

Criem:

```
produto := Produto{
    Nome: "Notebook",
    Preço: 3500,
}
```

E mostrem:

```
fmt.Println(produto.Nome)
fmt.Println(produto.Preço)
```

Quando isso estiver claro, pensem:

Se consigo representar um produto com uma struct, também consigo representar um bloco.

Por exemplo:

```
type Block struct {
    Index      int
    Timestamp  string
    Data       string
    PreviousHash string
    Hash       string
    Nonce      int
}
```

Não se preocupem ainda em preencher todos esses campos.

Primeiro entendam a estrutura.

Leitura recomendada

<https://go.dev/tour/moretypes/2>

4. Segunda dica: revisem Slices

Não teremos apenas um bloco.

Teremos:

```
Bloco 0
Bloco 1
Bloco 2
Bloco 3
Bloco 4
...
```

Portanto precisamos armazenar vários Block.

Para isso utilizaremos um slice:

```
var blockchain []Block
```

Revisem principalmente:

```
append()
```

Por exemplo:

```
blockchain = append(blockchain, novoBloco)
```

Isso significa:

```
blockchain
├── Block 0
├── Block 1
├── Block 2
└── novoBloco
```

Também revisem:

```
for _, bloco := range blockchain {
    fmt.Println(bloco)
}
```

Vocês precisarão disso para mostrar todos os blocos.

Leitura recomendada

<https://go.dev/tour/moretypes/7>

5. Terceira dica: revisem Ponteiros

Em determinado momento teremos uma função semelhante a:

```
func mineBlock(block *Block) {
```

Esse *Block será importante.

Durante a mineração precisaremos alterar:

```
Nonce
Hash
Tentativas
Tempo de mineração
```

do bloco que recebemos.

Por isso recomendo que antes façam:

```
package main

import "fmt"

func mudar(numero *int) {
    *numero = 100
}

func main() {

    x := 10

    mudar(&x)

    fmt.Println(x)
}
```

O resultado será:

```
100
```

Se vocês entenderem por que x mudou, já entenderam o suficiente sobre ponteiros para começar nosso projeto.

Leitura recomendada

<https://go.dev/tour/moretypes/1>

6. Quarta dica: aprendam a gravar arquivos

Essa parte é fundamental.

Não comecem tentando salvar uma blockchain.

Primeiro salvem apenas:

```
Olá Blockchain
```

em:

```
teste.txt
```

Pesquisem o pacote:

os

<https://pkg.go.dev/os>

Procurem principalmente:

```
os.WriteFile()  
os.ReadFile()  
os.Stat()  
os.IsNotExist()
```

Façam primeiro:

```
dados := []byte("Olá Blockchain")  
  
err := os.WriteFile(  
    "teste.txt",  
    dados,  
    0644,  
)
```

Depois tentem recuperar:

```
dados, err := os.ReadFile("teste.txt")
```

E mostrem:

```
fmt.Println(string(dados))
```

Quando conseguirem:

```
Go  
↓  
teste.txt  
↓  
fecha programa  
↓  
abre programa  
↓
```

```
Go recupera teste.txt
```

vocês estão prontos para a próxima etapa.

7. Quinta dica: entendam JSON

Nossa blockchain será salva em:

```
ledger.json
```

Por isso precisamos converter nossas structs para JSON.

Vocês precisam entender principalmente duas operações:

```
Marshal
```

e:

```
Unmarshal
```

Guardem:

```
MARSHAL
```

```
Go
```

```
↓
```

```
JSON
```

Enquanto:

```
UNMARSHAL
```

```
JSON
```

```
↓
```

```
Go
```

Leituras recomendadas

<https://go.dev/blog/json>

<https://pkg.go.dev/encoding/json>

Procurem principalmente:

```
json.Marshal()
json.MarshalIndent()
json.Unmarshal()
```

8. Façam um exercício de JSON antes da blockchain

Criem:

```
type Pessoa struct {
    Nome  string `json:"nome"`
    Idade int   `json:"idade"`
}
```

Depois:

```
peessoa := Pessoa{
    Nome:  "Carlos",
    Idade: 25,
}
```

Transformem isso em JSON.

O resultado esperado será parecido com:

```
{
  "nome": "Carlos",
  "idade": 25
}
```

Depois façam o caminho contrário.

Leiam o JSON e transformem novamente em:

Pessoa

Se vocês conseguirem fazer:

```
Struct
  ↓
JSON
  ↓
Arquivo
  ↓
JSON
  ↓
Struct
```

vocês resolveram boa parte do problema de persistência da nossa blockchain.

9. Sexta dica: façam um pequeno ledger antes da blockchain

Antes de trabalhar com blocos, criem:

```
type Registro struct {
    ID    int    `json:"id"`
    Texto string `json:"texto"`
}
```

Depois:

```
var registros []Registro
```

Adicionem três registros:

```
Registro 1
Registro 2
Registro 3
```

Salvem em:

```
ledger.json
```

Fechem o programa.

Executem novamente.

Tentem recuperar:

Registro 1
Registro 2
Registro 3

Esse exercício é extremamente importante.

Estamos fazendo:

Memória RAM
↓
JSON
↓
Disco

e posteriormente:

Disco
↓
JSON
↓
Memória RAM

Nossa blockchain utilizará exatamente essa lógica.

10. Sétima dica: agora estudem Hash

Somente agora eu recomendo entrar na parte de criptografia.

Vamos utilizar:

SHA-256

Não precisamos implementar o algoritmo matemático.

O Go já fornece:

```
import "crypto/sha256"
```

Leitura recomendada

<https://pkg.go.dev/crypto/sha256>

Procurem:

Sum256

Façam:

```
texto := "Curso de Go"  
  
hash := sha256.Sum256([]byte(texto))  
  
fmt.Printf("%x\n", hash)
```

Guardem o resultado.

Depois alterem:

Curso de Go

para:

Curso de GO

Executem novamente.

Vocês verão um hash completamente diferente.

11. Façam um teste de adulteração

Tentem:

João pagou 100

Calculem o hash.

Depois:

João pagou 10000

Calculem novamente.

Teremos conceitualmente:

"João pagou 100"

↓

SHA-256

↓

Hash A

Mas:

"João pagou 10000"

↓

SHA-256

↓

Hash B

Onde:

Hash A != Hash B

Pergunta que vocês precisam saber responder:

Como podemos utilizar essa propriedade para descobrir se alguém alterou um bloco?

Se vocês conseguirem responder, entenderam uma das ideias fundamentais do nosso projeto.

12. Agora podemos começar a pensar em Blockchain

Imaginem três blocos.

O primeiro possui:

BLOCO 0

Hash:

AAA

O segundo:

BLOCO 1

PreviousHash:
AAA

Hash:
BBB

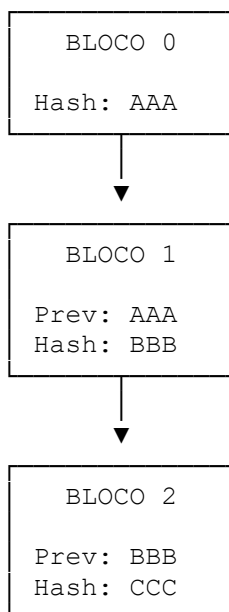
E o terceiro:

BLOCO 2

PreviousHash:
BBB

Hash:
CCC

Temos:



Agora pensem:

O que acontece se alguém alterar os dados do Bloco 1?

O hash dele muda.

Consequentemente o PreviousHash armazenado no Bloco 2 deixa de corresponder ao hash correto do Bloco 1.

A corrente foi quebrada.

13. Leitura introdutória sobre Blockchain e Bitcoin

Agora que vocês já entenderam hash e encadeamento, vale estudar como isso aparece em uma blockchain real.

Comecem pela explicação introdutória:

How does Bitcoin work?

<https://bitcoin.org/en/how-it-works>

Não tentem entender tudo.

Procurem entender:

Blockchain
Transactions
Mining
Confirmation

Façam anotações desses quatro conceitos.

14. Leiam uma parte do Bitcoin Whitepaper

Depois recomendo conhecer o documento original:

Bitcoin: A Peer-to-Peer Electronic Cash System

<https://bitcoin.org/bitcoin.pdf>

Não estou pedindo que vocês dominem o whitepaper inteiro.

Para este projeto, concentrem-se principalmente em:

1. Introduction
2. Transactions
3. Timestamp Server
4. Proof-of-Work

O mais importante para nosso projeto é a seção:

4. Proof-of-Work

15. Oitava dica: façam Proof of Work separado

Não coloquem Proof of Work diretamente na blockchain.

Primeiro criem um programa separado.

Nosso objetivo será encontrar um hash começando com:

0000

Imagem:

Nonce = 0
↓
Calcula hash
↓
87adf3...
↓
Não começa com 0000

Tentamos:

Nonce = 1

Depois:

Nonce = 2

E assim por diante.

Até:

Nonce = 48372

↓

Hash

↓

00008af91...

↓

ENCONTRAMOS!

16. Criem seu primeiro minerador

Uma boa experiência seria:

```
package main

import (
    "crypto/sha256"
    "fmt"
    "strconv"
    "strings"
)

func main() {
    dados := "Meu primeiro bloco"

    nonce := 0

    alvo := "0000"

    for {
        texto := dados + strconv.Itoa(nonce)

        hash := sha256.Sum256([]byte(texto))

        hashTexto := fmt.Sprintf("%x", hash)

        if strings.HasPrefix(hashTexto, alvo) {
            fmt.Println("Hash encontrado!")

            fmt.Println("Nonce:", nonce)

            fmt.Println("Hash:", hashTexto)

            break
        }

        nonce++
    }
}
```

Tentem entender cada linha.
Não simplesmente executem.
Principalmente:

```
for {  
  
    nonce++  
  
e:  
    strings.HasPrefix()
```

17. Experimentem diferentes dificuldades

Façam o alvo ser:

0

Depois:

00

Depois:

000

Depois:

0000

E finalmente:

00000

Anotem:

```
Dificuldade  
Tentativas  
Tempo  
Nonce
```

Vocês deverão perceber que aumentar a dificuldade aumenta, em média, a quantidade de trabalho necessária.

Essa é uma excelente experiência prática para compreender o conceito de Proof of Work.

18. Agora criem o Genesis Block

Toda nossa blockchain precisa começar em algum lugar.

Vamos chamar o primeiro bloco de:

Genesis Block

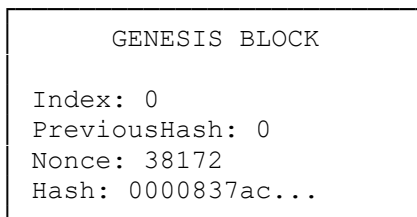
Ele será:

Index: 0

Como não existe bloco anterior:

```
PreviousHash: 0
```

Teremos:



Façam primeiro apenas o Genesis Block.

Não tentem criar dez blocos ainda.

19. Criem o segundo bloco

Agora criem:

```
Bloco 1
```

A principal regra será:

```
PreviousHash = genesisBlock.Hash
```

Teremos:

```
Genesis  
Hash AAA  
|  
▼  
Bloco 1  
PreviousHash AAA  
Hash BBB
```

Quando isso funcionar, criem o Bloco 2.

Depois o Bloco 3.

20. Agora coloquem os blocos em um Slice

Em vez de:

```
block0  
block1  
block2  
block3
```

utilizem:

```
var blockchain []Block
```

Adicionem:

```
blockchain = append(blockchain, bloco)
```

Agora teremos:

```
blockchain
├── Block 0
├── Block 1
├── Block 2
├── Block 3
└── Block 4
```

Percorram usando:

```
for _, block := range blockchain {
```

e mostrem todos os blocos.

21. Criem a validação

Agora vocês precisam responder:

Como descobrir se alguém alterou nossa blockchain?

Comecem verificando:

```
Hash armazenado
    =
Hash recalculado?
```

Depois:

```
PreviousHash do bloco atual
    =
Hash do bloco anterior?
```

Finalmente verifiquem:

Hash começa com 0000?

Nosso algoritmo será aproximadamente:

```
Pegar bloco
    ↓
Recalcular hash
    ↓
Hash está correto?
    ↓
PreviousHash está correto?
    ↓
Proof of Work está correto?
    ↓
SIM
    ↓
Próximo bloco
```

Se algum teste falhar:

BLOCKCHAIN INVÁLIDA

22. Agora juntem Blockchain + JSON + Arquivos

Chegamos à persistência.

Quando a blockchain estiver na memória:

```
[]Block
```

façam:

```
[]Block
  ↓
json.MarshalIndent()
  ↓
JSON
  ↓
os.WriteFile()
  ↓
ledger.json
```

Quando o programa iniciar:

```
ledger.json
  ↓
os.ReadFile()
  ↓
JSON
  ↓
json.Unmarshal()
  ↓
[]Block
```

Esse é o coração da persistência do projeto.

23. Façam o teste mais importante

Criem:

```
Genesis
  ↓
Bloco 1
  ↓
Bloco 2
  ↓
Bloco 3
```

Agora:

```
FECHEM O PROGRAMA
```

Executem novamente:

```
go run .
```

Se aparecer:

```
Blockchain carregada.
```

```
Blocos encontrados: 4
```

vocês estão no caminho certo.

Agora adicionem outro bloco.

O resultado precisa ser:

Genesis

↓

Bloco 1

↓

Bloco 2

↓

Bloco 3

PROGRAMA FOI FECHADO

PROGRAMA FOI ABERTO

↓

Bloco 4

Se o programa criar outro Genesis Block, existe um erro na lógica de inicialização.

24. Adicionem as Transações

Agora evoluam o projeto.

Criem:

```
type Transaction struct {  
    From    string  
    To      string  
    Amount  float64  
}
```

Uma transação poderá representar:

João
↓ R\$ 500
▼
Maria

Depois façam o bloco armazenar:

```
Transactions []Transaction
```

Assim um único bloco poderá possuir:

BLOCO 5

Transação 1

João → Maria R\$500

Transação 2

Carlos → Ana R\$150

Transação 3

Pedro → João R\$75

25. Criem as transações pendentes

Não coloquem uma nova transação imediatamente na blockchain.

Criem:

```
var pendingTransactions []Transaction
```

Quando o usuário criar uma transação:

```
Nova transação
    ↓
Transações pendentes
    ↓
Aguardando mineração
```

Somente depois:

```
Transações pendentes
    ↓
Minerador
    ↓
Proof of Work
    ↓
Bloco
    ↓
Blockchain
```

Essa separação é importante.

26. Adicionem o nome do minerador

Façam o programa perguntar:

```
Nome do minerador:
```

Adicionem ao bloco:

```
Miner string
```

Assim poderemos ter:

```
Bloco: 7
```

```
Minerador:
Antonio
```

```
Nonce:
48372
```

```
Hash:
000083ac...
```

27. Meçam o tempo de mineração

Antes de minerar:

```
inicio := time.Now()
```

Depois:

```
duracao := time.Since(inicio)
```

Mostrem:

Dificuldade: 4

Tentativas:

87.321

Nonce:

87.320

Tempo:

53ms

Hash:

00008371ac...

Façam experiências alterando a dificuldade.

28. Criem o menu somente no final

Não recomendo começar pelo menu.

Primeiro façam as funções funcionarem.

Somente depois criem:

```
=====
                        BLOCKCHAIN EM GO
=====
```

```
1 - Criar nova transação
2 - Minerar bloco
3 - Mostrar blockchain
4 - Validar blockchain
5 - Mostrar último bloco
6 - Informações da blockchain
7 - Mostrar transações pendentes
0 - Sair
```

O menu deve apenas chamar funcionalidades que vocês já testaram separadamente.

29. Teste obrigatório: adulterem o Ledger

Depois que tudo funcionar:

1. Criem alguns blocos.
2. Fechem o programa.
3. Abram `ledger.json`.
4. Alterem manualmente uma transação.

Por exemplo:

```
"amount": 500
```

para:

```
"amount": 500000
```

Salvem.

Executem novamente:

```
go run .
```

O programa deverá detectar que a blockchain foi modificada.

O resultado esperado será algo semelhante a:

```
ERRO!
```

```
BLOCKCHAIN INVÁLIDA!
```

```
Hash inválido no bloco 2.
```

Se o programa aceitar normalmente a alteração, revisem a função de validação.

30. Ordem que eu recomendo para vocês

Não façam:

```
Projeto completo
```

↓

```
Tentar descobrir por que não funciona
```

Façam:

```
1. Struct
```

↓

```
2. Slice
```

↓

```
3. Arquivo
```

↓

```
4. JSON
```

↓

```
5. Salvar e recuperar JSON
```

↓

```
6. SHA-256
```

↓

```
7. Genesis Block
```

↓

```
8. PreviousHash
```

↓

```
9. Blockchain
```

↓

```
10. Proof of Work
```

↓

```
11. Validação
```

↓

```
12. Ledger
```

↓

```
13. Persistência
```

↓

```
14. Transações
```

↓

```
15. Mineração
```

↓

```
16. Menu
```

Façam uma etapa funcionar antes de passar para a próxima.

31. Leituras que recomendo

Não é necessário estudar dezenas de livros ou sites. Para este projeto, concentrem-se nestes materiais.

Go: primeiros passos

<https://go.dev/doc/tutorial/getting-started>

Leiam para revisar:

Módulos
go mod
go run
Estrutura inicial

Tour de Go

<https://go.dev/tour/>

Procurem:

Functions
Structs
Slices
Pointers
For
Range
Errors

Arquivos

<https://pkg.go.dev/os>

Procurem:

ReadFile
WriteFile
Stat
IsNotExist

JSON

<https://go.dev/blog/json>

e:

<https://pkg.go.dev/encoding/json>

Procurem:

Marshal
MarshalIndent
Unmarshal

SHA-256

<https://pkg.go.dev/crypto/sha256>

Procurem:

Sum256

Introdução ao Bitcoin

<https://bitcoin.org/en/how-it-works>

Procurem entender:

Transactions

Blockchain

Mining

Confirmation

Bitcoin Whitepaper

<https://bitcoin.org/bitcoin.pdf>

Para este trabalho, concentrem-se inicialmente nas seções:

1. Introduction
2. Transactions
3. Timestamp Server
4. Proof-of-Work

Não precisam dominar o restante para realizar este projeto.

32. Checklist antes de começar o código final

Antes de integrar tudo, quero que vocês consigam responder:

- O que é uma `struct`?
- Para que serve um `slice`?
- Para que utilizamos `append()`?
- Como percorremos um `slice` com `range`?
- Para que serve um ponteiro?
- Como tratamos um `error` em Go?
- O que `os.WriteFile()` faz?
- O que `os.ReadFile()` faz?
- O que significa `Marshal`?
- O que significa `Unmarshal`?
- O que é SHA-256?
- O que é um `hash`?

- O que é um bloco?
- O que é o Genesis Block?
- Para que serve PreviousHash?
- O que é um Nonce?
- O que é Proof of Work?
- O que significa minerar nosso bloco?
- Para que serve ledger.json?
- Como detectamos uma alteração em um bloco?
- Como recuperamos a blockchain depois que o programa é reiniciado?

Se vocês conseguirem explicar esses pontos com suas próprias palavras, estarão preparados para montar o programa final.

33. O que não quero que vocês se preocupem agora

Nosso projeto é uma **blockchain educacional**.

Não tentem adicionar neste momento:

Smart Contracts
 Tokens
 Proof of Stake
 Merkle Trees
 Carteiras
 Chaves privadas
 ECDSA
 Rede P2P
 Nós distribuídos
 Forks
 UTXO
 Gas
 Staking

Esses assuntos são interessantes, mas aumentariam muito a complexidade.

Nosso objetivo agora é dominar:

Dados
 ↓
 Transação
 ↓
 Bloco
 ↓
 Hash
 ↓
 PreviousHash
 ↓
 Nonce
 ↓
 Proof of Work
 ↓
 Blockchain
 ↓

Ledger
↓
Persistência

34. Sugestão de organização em sete dias

Se vocês quiserem organizar o projeto ao longo de uma semana, sugiro:

Dia 1

Revisem:

Structs
Slices
Ponteiros
Funções

Façam pequenos programas.

Dia 2

Estudem:

ReadFile
WriteFile
JSON

Façam um programa salvar e recuperar informações.

Dia 3

Criem um pequeno:

ledger.json

Fechem e reabram o programa e confirmem que os dados continuam existindo.

Dia 4

Estudem:

SHA-256
Hash
PreviousHash

Criem:

Genesis → Bloco 1 → Bloco 2

Dia 5

Implementem:

Nonce
Difficulty
Proof of Work
Mineração

Dia 6

Implementem:

Validação
Transações
Transações pendentes
Persistência

Dia 7

Juntem tudo e criem:

Menu
+
Transações
+
Mineração
+
Blockchain
+
Validação
+
Ledger
+
Persistência

35. Como vou considerar que o projeto funcionou

No mínimo, quero conseguir executar:

```
go run .
```

Criar algumas transações:

João → Maria → R\$500

Carlos → Pedro → R\$150

Minerar um bloco:

Minerando...

Tentativas: 48732
Nonce: 48731
Hash: 00008372ac...

Criar vários blocos:

Genesis
↓
Bloco 1
↓
Bloco 2
↓
Bloco 3

Fechar o programa.

Executar novamente:

```
go run .
```

E continuar:

```
Genesis
  ↓
Bloco 1
  ↓
Bloco 2
  ↓
Bloco 3
  ↓
Bloco 4
```

Também quero conseguir alterar manualmente uma transação antiga no `ledger.json` e ver o programa detectar:

```
BLOCKCHAIN INVÁLIDA
```

Se vocês conseguirem demonstrar essas duas situações, **persistência e detecção de adulteração**, já terão demonstrado os conceitos mais importantes do nosso projeto.

36. Dica final do professor

Não tenham pressa para chegar ao código completo.

Quando um projeto parece complicado, normalmente o problema não precisa ser resolvido inteiro de uma vez.

Quebrem:

```
"Preciso construir uma blockchain"
```

em problemas menores:

```
Como crio um bloco?
```

```
Como calculo um hash?
```

```
Como ligo dois blocos?
```

```
Como encontro um nonce?
```

```
Como valido um bloco?
```

```
Como salvo os blocos?
```

```
Como recupero os blocos?
```

```
Como adiciono uma transação?
```

Cada pergunta dessas é um pequeno programa que vocês já conseguem resolver utilizando o que aprenderam em Go.

Quando todas essas pequenas partes estiverem funcionando, basta integrá-las.

E esse é talvez o aprendizado mais importante deste projeto final: **um programa grande é normalmente formado por vários problemas pequenos que conseguimos resolver separadamente.**